The Little Verser

A friendly dialogue about Verse,
the language that knows how to fail

Questions on the left. Answers on the right.
Try not to peek.

The openverse project

November 2026

Contents

Before We Begin **1**

1 Plain Values **2**

2 Names and Recipes **9**

3 Yes, No, and Fail **17**

4 Take It Back **24**

5 Again and Again **30**

6 Maybe **37**

7 Shapes of Things **42**

8 Promises Kept **48**

A The Rules **56**

Before We Begin

This little book is a conversation. On the left of each page is a question; on the right, its answer. Cover the right side with a card, read a question, and answer it yourself before you slide the card down. Some answers are a single word. Some are a program's output. A few are "No!", and those are usually the most interesting.

The language is **Verse**. Epic Games made it for building games, and it has an idea at its heart that most languages do not: an expression can *fail*. Failing is not crashing, and it is not the same as `false`. A failure is a polite "no": the program notices it, steps back, and tries something else, and anything the failed attempt did is quietly undone. By the end of this book you will think in failures, and you will like it.

We use **openverse**, a small, careful implementation of Verse. It runs a deterministic part of the language: the same program always gives the same answer. The people who built openverse proved, with a computer checking every step, that it keeps the promises we meet in the last chapter.

Every program in this book is real. Each one was run by openverse when the book was made, and the output you see is what it printed. To run one yourself, save it in a file, say `hunt.verse`, and type

```
openverse run hunt.verse
```

or paste it into the openverse playground, which runs openverse right in your web browser.

You need no mathematics beyond arithmetic, and no programming. You need a little patience, a pencil, and the card.

Do not hurry. Each chapter builds on the one before. When a box with a **Rule** appears, stop and read it twice: the rules are the whole book in miniature, and the last pages collect them all.

This book shows the part of Verse that openverse runs, and Verse is bigger than that. When you want the whole language, read the *Book of Verse*, Epic Games' own guide to it. Thank you, Epic Games, for making Verse and for sharing it with everyone. (openverse is an independent project: Epic Games did not make it and does not endorse it. Verse and Fortnite are trademarks of Epic Games.)

Ready? Let's go treasure hunting.

1

Plain Values

A treasure hunt needs treasure, and a program needs *values*: numbers, words, true-or-false answers, and collections of them. In this chapter we meet the values of Verse, and the one way a program shows them to us.

Is `42` a value?

Yes. It is a whole number. Verse calls whole numbers `int`.

Is `-7` an `int`?

Yes. An `int` may be negative.

Is `0` an `int`?

Yes, and a very useful one.

Is `3.5` an `int`?

No. An `int` has no fraction.

What is `3 + 4`?

`7`.

What is `2 * 3 + 1`?

`7` again. Multiplication goes first, as in school.

What is `2 * (3 + 1)`?

`8`. Parentheses go first of all.

What is `10 - 20`?

`-10`.

How do we see a value?

A program *prints* it.

What does this program print?

```
Main():void =
    Print("Hello, treasure hunter!")
```

Run in the playground

```
Hello, treasure hunter!
```

What is `Main`?

The place where a program starts. openverse runs `Main` first.

What is `Print`?

It writes some text on a line of its own.

What is `"Hello, treasure hunter!"`?

A *string*: text between double quotes.

Is `"42"` an `int`?

No. It is a string with two characters in it, `4` and `2`.

Can we `Print` an `int`?

```
Main():void =
    Print(42)
```

Run in the playground

Rejected: `expected string, found int`

No. `Print` wants a string, and `42` is an `int`.

That is a kind message. What does it say?

That `Print` *expected* a string but *found* an `int`.

When does openverse tell us this?

Before the program runs at all. It reads the whole program first.

Then how do we print `42`?

```
Main():void =
    Print("{42}")
```

Run in the playground

```
42
```

What do the braces `{` and `}` do?

They hold a value. Its text is put into the string at that spot.

What does this print?

```
Main():void =
    Print("I have {3 + 4} coins.")
```

Run in the playground

```
I have 7 coins.
```

Was `3 + 4` printed?

No. Its value, `7`, was.

What is `3.5`, if it is not an `int`?

A `float`: a number with a point. `2.0` is one too.

What does this print?

```
Main():void =
    Print("{1.5 + 1.5}")
```

Run in the playground

```
3.0
```

A `float` keeps its point, even when it is whole.

And this?

```
Main():void =
    Print("{0.1 + 0.2}")
```

Run in the playground

```
0.30000000000000004
```

Almost `0.3`. A `float` is a close guess.

Then when do we use an `int`?

For what we count: coins, gems, keys. A `float` is for what we measure.

What does this print?

```
Main():void =
    Print("one")
    Print("two")
    Print("three")
```

Run in the playground

```
one
two
three
```

In what order?

From top to bottom, one line each.

Why are the three `Print`s pushed to the right?

They are the *body* of `Main`. Everything indented under `Main():void =` belongs to it.

The Rule of the Braces

To show a value, put it in braces
inside a string, and print the string.

Is `true` a value?

Yes.

Is `false` a value?

Yes. `true` and `false` are the two values of type `logic`.

Can we write `Print("{true}")`?

```
Main():void =
    Print("{true}")
```

Run in the playground

Rejected: `no ToString for logic`

No. openverse knows how to write numbers, characters and strings as text, but not `logic`.

So what is `logic` good for?

For asking questions. We will ask a lot of them in Chapter 3.

Is `'g'` a value?

Yes. It is a single *character*, between single quotes. Its type is `char`.

Is `'g'` the same as `"g"`?

No. `"g"` is a string that holds one character.

What does this print?

```
Main():void =
    Print("Found a {'g'}em!")
```

Run in the playground

```
Found a gem!
```

Can a string be empty?

Yes: `""`.

What is `"gold" + "en"`?

`"golden"`. `+` sticks strings together.

What does this print?

```
Main():void =
    Print("treasure" + " " + "map")
```

Run in the playground

```
treasure map
```

What is `array{3, 1, 4}`?

An *array*: three `int`s in a row.

What is its type?

`[]int`, said "array of `int`".

What is `array{3, 1, 4}.Length`?

`3`. It has three elements.

What does this print?

```
Main():void =
    Coins := array{5, 10, 25}
    Print("{Coins.Length} coins")
```

Run in the playground

```
3 coins
```

What is `Coins`?

A *name* for the array. We will name many things in Chapter 2.

What is `"gem".Length`?

```
Main():void =
    Print("{"gem".Length}")
```

Run in the playground

```
3
```

Why does a string have a `Length`?

Because a string *is* an array: an array of `char`s. Its type, `string`, is another name for `[]char`.

Can an array hold an `int` and a string?

```
Main():void =
    Mixed := array{1, "two"}
    Print("{Mixed.Length}")
```

Run in the playground

```
2
```

Yes, but only as things of type `any`, the type every value has.

What can we do with an `any`?

```
Main():void =
    Mixed := array{1, "two"}
    if (X := Mixed[0]):
        Print("{X + 1}")
```

Run in the playground

Rejected: `no operator '+' for any and int`

Almost nothing. Verse no longer knows that `Mixed[0]` is the `int` `1`. Keep one kind of thing in an array.

Is `array{}` an array?

Yes, the empty one. Its `Length` is `0`.

What if we want an `int` and a string together?

We use a *tuple*: `(1, "two")`.

What is the type of `(1, "two")`?

`tuple(int, string)`.

How do we take a tuple apart?

```
Main():void =
    Prize := (100, "gold")
    Print("{Prize(0)} pieces of {Prize(1)}")
```

Run in the playground

```
100 pieces of gold
```

Why `Prize(0)` and not `Prize(1)` for the first part?

Counting starts at zero: the first part is part `0`.

Is a tuple an array?

No. A tuple has a fixed number of parts, each with its own type.

The Rule of Types

Every value has a type: `int`, `float`, `logic`, `char`, `string`, an array type such as `[]int`, or a tuple type such as `tuple(int, string)`. An array's elements all share one type; a tuple's parts each have their own.

What is `void`?

The type of "nothing worth giving back".

What does `Main():void` say?

`Main` takes nothing (the empty parentheses) and gives back nothing of interest (`void`).

Then what does `Main` do?

It does things, such as printing.

Can we leave notes in a program?

```
Main():void =
    # This line is a comment.
    Print("Notes are for people.")
```

Run in the playground

```
Notes are for people.
```

What happened to the line that starts with #?

openverse skipped it. Everything after # on a line is a *comment*.

What does this print?

```
Main():void =
    Gems := array{'r', 'u', 'b', 'y'}
    Print("{Gems.Length} letters")
    Print("a " + "ruby")
```

Run in the playground

```
4 letters
a ruby
```

Is `Gems` a string?

Yes! An array of `chars` *is* a string.

Can we print `Gems` itself, then?

```
Main():void =
    Gems := array{'r', 'u', 'b', 'y'}
    Print(Gems)
```

Run in the playground

```
ruby
```

What does `Abs(-5)` give?

`5`: the size of a number, without its sign.

And `Min(3, 9)` and `Max(3, 9)`?

```
Main():void =
    Print("{Min(3, 9)} and {Max(3, 9)}")
```

Run in the playground

```
3 and 9
```

Have we met every kind of value?	No. Options, maps, structs and objects are still ahead. But these are the plain ones.

That is enough treasure for one chapter. We have values; next we give them names, and teach Verse some recipes of our own.

2

Names and Recipes

A treasure hunter keeps notes: how many coins, which chest, what the map said. In Verse we write notes by giving values *names*. Then we write *recipes*, called functions, that turn values into other values.

What does this program print?

```
Main():void =
    Coins := 5
    Print("{Coins} coins")
```

Run in the playground

```
5 coins
```

What is `Coins`?

A *name*. Here it stands for the value `5`.

What does `:=` say?

"From now on, this name stands for this value."

Can we use a name in arithmetic?

```
Main():void =
    Coins := 5
    Gems := 2
    Print("{Coins + Gems * 10} points")
```

Run in the playground

```
25 points
```

Can we say which type a name has?

```
Main():void =
    Coins:int = 5
    Print("{Coins + 1} coins")
```

Run in the playground

```
6 coins
```

How is `Coins:int = 5` different from `Coins := 5`?

Only in that we wrote the type, `int`, ourselves. With `:=`, Verse works it out.

What if the type we write is wrong?

```
Main():void =
    Coins:int = "five"
```

Run in the playground

Rejected: `expected int, found string`

```
Main():void =
    Coins := 5
    Coins := 6
```

Run in the playground

Rejected: `'Coins' is already defined`

```
6 coins
```

```
Main():void =
    var Coins:int = 5
    set Coins = 6
    Print("{Coins} coins")
```

Run in the playground

```
Main():void =
    Coins := 5
    set Coins = 6
```

Run in the playground

Rejected: `'Coins' is not a var`

```
Main():void =
    var Coins:int = 5
    set Coins += 10
    set Coins -= 3
    set Coins *= 2

    Print("{Coins}")
```

Run in the playground

```
24
```

The Rule of Names

A name stands for one value for as long as it lives. Only a `var` may change, and only by `set`.

```
42
```

```
Double(N:int):int = N * 2

Main():void =
    Print("{Double(21)}")
```

Run in the playground

```
25
```

```
Score(Coins:int, Gems:int):int =
    Coins + Gems * 10

Main():void =
    Print("{Score(5, 2)}")
```

Run in the playground

```
25
```

```
Score(Coins:int, Gems:int):int =
    Coins + Gems * 10

Main():void =
    Loot := (5, 2)
    Print("{Score(Loot)}")
```

Run in the playground

```
28
```

```
Score(Coins:int, Gems:int):int =
    GemPoints := Gems * 10
    Bonus := 3
    Coins + GemPoints + Bonus

Main():void =
    Print("{Score(5, 2)}")
```

Run in the playground

```
Welcome, Ada!
Welcome, Lin!
```

```
Greet(Name:string):string =
    "Welcome, {Name}!"

Main():void =
    Print(Greet("Ada"))
    Print(Greet("Lin"))
```

Run in the playground

```
Shout(Word:string):void =
    Print(Word + "!")

Main():void =

    Shout("Treasure")
```

Run in the playground

```

Treasure!
```

```
Countdown(From:int):void =
    var N:int = From
    Print("{N}")
    set N -= 1
    Print("{N}")
    set N -= 1
    Print("{N}")

Main():void =
    Countdown(3)
```

Run in the playground

```
3
2
1
```

```
GemValue:int = 10

Worth(Gems:int):int = Gems * GemValue

Main():void =
    Print("{Worth(3)}")
```

Run in the playground

```
30
```

```
Main():void =
    Prize := 5
    if (Prize > 3):
        Prize := 9
        Print("{Prize}")
```

Run in the playground

Rejected: `already defined`

```
Describe(N:int):string = "{N} coins"
Describe(S:string):string = "a {S}"

Main():void =
    Print(Describe(3))
    Print(Describe("gem"))
```

Run in the playground

```
3 coins
a gem
```

```
Describe(N:int):string = "{N} coins"
Describe(K:int):string = "{K} keys"

Main():void =
    Print(Describe(3))
```

Run in the playground

Rejected: `a call cannot tell apart`

```
Double(N:int):int = N * 2
Twice(F:int->int, N:int):int = F(F(N))

Main():void =
    Print("{Twice(Double, 5)}")
```

Run in the playground

```
20
```

```
Double(N:int):int = N * 2
AddOne(N:int):int = N + 1

Main():void =
    var Step:int->int = Double
    Print("{Step(10)}")
    set Step = AddOne
    Print("{Step(10)}")
```

Run in the playground

```
20
11
```

```
Shout(S:string):string = "{S}!"
Twice(F:int->int, N:int):int = F(F(N))

Main():void =
    Print("{Twice(Shout, 5)}")
```

Run in the playground

Rejected: `expected type{_(:int):int}, found type{_(:string):string}`

```
3 gems
```

```
treasure := module:
    Gems<public>:int = 3
    Show<public>(N:int):string = "{N} gems"

Main():void =
    Print(treasure.Show(treasure.Gems))
```

Run in the playground

```
treasure := module:
    Gems:int = 3

Main():void =
    Print("{treasure.Gems}")
```

Run in the playground

Rejected: `is internal to treasure`

```
3 gems
```

```
treasure := module:
    Gems<public>:int = 3

using{treasure}

Main():void =
    Print("{Gems} gems")
```

Run in the playground

The Rule of Recipes

A function names its parameters, says what type it gives back, and gives back the value of its last line. A `void` function is used for what it does.

3

Yes, No, and Fail

```
we can buy the map
```

```
Main():void =
    Coins := 12
    if (Coins > 10):
        Print("we can buy the map")
```

Run in the playground

```
Main():void =
    Coins := 7
    if (Coins > 10):
        Print("we can buy the map")
    Print("done")
```

Run in the playground

```
done
```

```
Main():void =
    Coins := 7
    if (Coins > 10):
        Print("we can buy the map")
    else:
        Print("we need more coins")
```

Run in the playground

```
we need more coins
```

```
Main():void =
    Coins := 7
    if (Coins > 10) then Print("buy")
    else Print("save")
```

Run in the playground

```
save
```

```
Main():void =
    if (3 = 3) { Print("equal") }
    if (3 <> 4) { Print("not equal") }
    if ("gem" = "gem") { Print("same word") }
```

Run in the playground

```
equal
not equal
same word
```

```
Main():void =
    Coins := 7
    Coins > 10
    Print("ok")
```

Run in the playground

```
Rejected: this expression can fail, but
it is not in a failure context
```

The Rule of Failure

A question in Verse succeeds or fails. Anything that might fail must sit in a failure context, such as the condition of an `if`.

```
a digit
```

```
Main():void =
    X := 5
    if (0 <= X <= 9) { Print("a digit") }
```

Run in the playground

```
Y is 5
```

```
Main():void =
    X := 5
    if (Y := (X > 3)) { Print("Y is {Y}") }
```

Run in the playground

```
Main():void =
    Coins := 12
    Keys := 1
    if (Coins > 10, Keys > 0):
        Print("map bought, gate open")
```

Run in the playground

```
map bought, gate open
```

```
Main():void =
    Lit:logic = true
    if (Lit?) { Print("the torch is lit") }
    Dark:logic = false
    if (Dark?):
        Print("dark?")
    else:
        Print("not dark")
```

Run in the playground

```
the torch is lit
not dark
```

```
Main():void =
    Coins := 12
    Size := if (Coins > 10):
        "rich"
    else:
        "poor"
    Print(Size)
```

Run in the playground

```
rich
```

```
Main():void =
    if (1 + 1) { Print("two") }
```

Run in the playground

Rejected: `cannot fail`

The Rule of the Question Mark

`false` is a value, not a failure. To turn a `logic` into a question, add `?`: `X?` succeeds when `X` is `true` and fails when it is `false`.

```
Main():void =
    Chests := array{10, 20, 30}
    if (A := Chests[1]):
        Print("chest 1 holds {A}")
    if (B := Chests[3]):
        Print("chest 3 holds {B}")
    else:
        Print("there is no chest 3")
```

Run in the playground

```
chest 1 holds 20
there is no chest 3
```

```
Main():void =
    if (H := 7 / 2):
        Print("{Floor(H)} to {Ceil(H)}")
```

Run in the playground

```
3 to 4
```

```
IsEven(N:int)<decides>:int = Mod[N, 2] = 0

Main():void =
    if (IsEven[4]) { Print("4 is even") }
    if (IsEven[7]):
        Print("7 is even?")
    else:
        Print("7 is odd")
```

Run in the playground

```
4 is even
7 is odd
```

```
IsEven(N:int)<decides>:int = Mod[N, 2] = 0

Main():void =
    if (IsEven(4)) { Print("even") }
```

Run in the playground

Rejected: `call it with IsEven[`

```
half of 10 is 5
7 has no half
```

```
Half(N:int)<decides>:int =
    Mod[N, 2] = 0
    Quotient[N, 2]

Main():void =
    if (H := Half[10]):
        Print("half of 10 is {H}")
    if (H := Half[7]):
        Print("{H}")
    else:
        Print("7 has no half")
```

Run in the playground

The Rule of Brackets

A function that may fail is marked `<decides>` and is called with square brackets, `F[X]`. A function without `<decides>` is called with parentheses, `F(X)`, and never fails.

4

Take It Back

```
still have 10 coins
```

```
Main():void =
    var Coins:int = 10
    if (set Coins -= 25, Coins >= 0):
        Print("bought the map")
    else:
        Print("still have {Coins} coins")
```

Run in the playground

```
bought it; 5 left
```

```
Main():void =
    var Coins:int = 30
    if (set Coins -= 25, Coins >= 0):
        Print("bought it; {Coins} left")
```

Run in the playground

```
Main():void =
    var Torches:int = 3
    if (set Torches -= 1,
        Print("lighting a torch"),
        Torches > 5):
        Print("so bright")
    Print("torches: {Torches}")
```

Run in the playground

```
torches: 3
```

```
Main():void =
    var X:int = 1
    if (set X = 2, set X = 3, X = 4):
        Print("never")
    Print("X = {X}")
```

Run in the playground

```
X = 1
```

The Rule of the Safe Guess

When a question fails, everything it did is taken back: every `set` and every `Print`. When it succeeds, everything it did is kept.

```
7 is odd
```

```
IsEven(N:int)<decides>:int = Mod[N, 2] = 0

Main():void =
    if (not IsEven[7]) { Print("7 is odd") }
    if (not IsEven[8]) { Print("8 is odd") }
```

Run in the playground

```
Steps = 0
```

```
Main():void =
    var Steps:int = 0
    if (not (set Steps = 100; Steps > 50)):
        Print("never")
    else:
        Print("Steps = {Steps}")
```

Run in the playground

```
Half(N:int)<decides>:int =
    Mod[N, 2] = 0
    Quotient[N, 2]

Main():void =
    A := Half[7] or 0
    B := Half[8] or 0
    Print("{A} {B}")
```

Run in the playground

```
0 4
```

```
Half(N:int)<decides>:int =
    Mod[N, 2] = 0
    Quotient[N, 2]

Main():void =
    C := Half[7] or Half[9] or Half[12] or -1
    Print("{C}")
```

Run in the playground

```
6
```

```
Main():void =
    var Tries:int = 0
    Prize := (set Tries += 1; Tries > 5) or 99
    Print("prize {Prize}, tries {Tries}")
```

Run in the playground

```
prize 99, tries 0
```

```
not both
at least one
```

```
Main():void =
    Coins := 12
    Keys := 0
    if (Coins > 10 and Keys > 0):
        Print("both")
    else:
        Print("not both")

    if (Coins > 10 or Keys > 0):
        Print("at least one")
```

Run in the playground

```
Main():void =
    if ((X := 5) and X > 3):
        Print("x")
```

Run in the playground

Rejected: `unknown identifier 'X'`

The Rule of Not, Or and And

`not` succeeds when its inside fails, and never keeps its inside's work. `or` tries the left side, and if it fails, takes it back and tries the right. `and` needs both sides to succeed.

```
the chest opens
```

```
HasGold(Keys:int)<decides>:void =
    Keys >= 100
HasSilver(Keys:int)<decides>:void =
    Mod[Keys, 10] = 5

Main():void =
    Keys := 5
    if (HasGold[Keys] or HasSilver[Keys]):
        Print("the chest opens")
```

Run in the playground

```
Main():void =
    var Coins:int = 0
    if (set Coins -= 1, Coins >= 0):
        Print("opened")
    else:
        Print("no coin, no chest")
    Print("coins: {Coins}")
```

Run in the playground

```
no coin, no chest
coins: 0
```

5

Again and Again

```
Main():void =
    Gems := array{"ruby", "opal", "jade"}
    for (Gem : Gems):
        Print("found: {Gem}")
```

Run in the playground

```
found: ruby
found: opal
found: jade
```

```
Main():void =
    for (N := 1..4) { Print("{N}") }
```

Run in the playground

```
1
2
3
4
```

```
Main():void =
    Doubled := for (N : array{1, 2, 3}):
        N * 2
    Print("{Doubled.Length} values")
    if (Last := Doubled[2]):
        Print("last is {Last}")
```

Run in the playground

```
3 values
last is 6
```

```
Main():void =
    Evens := for (N := 1..10, Mod[N, 2] = 0):
        N
    Print("{Evens.Length} even numbers")
```

Run in the playground

```
5 even numbers
```

```
Main():void =
    Lucky := for (X := 1..20, Mod[X, 7] = 0):
        X * X
    for (L : Lucky) { Print("{L}") }
```

Run in the playground

```
49
196
```

```
Main():void =
    for (I -> Gem : array{"ruby", "opal"}):
        Print("{I}: {Gem}")
```

Run in the playground

```
0: ruby
1: opal
```

```
Main():void =
    for (Row := 1..2, Col := 1..3):
        Print("{Row},{Col}")
```

Run in the playground

```
1,1
1,2
1,3
2,1
2,2
2,3
```

```
Main():void =
    for (Gem := 1 | 5 | 10):
        Print("a gem worth {Gem}")
```

Run in the playground

```
a gem worth 1
a gem worth 5
a gem worth 10
```

```
Main():void =
    for (X := 1 | 2, Y := 10 | 20):
        Print("{X + Y}")
```

Run in the playground

```
11
21
12
22
```

```
Main():void =
    if (Gem := 1 | 5 | 10, Gem > 3):
        Print("we keep {Gem}")
```

Run in the playground

```
we keep 5
```

```
    if (Gem := 1 | 2, Gem > 3):
        Print("we keep {Gem}")
    Print("none worth keeping")
```

Run in the playground

```
none worth keeping
```

```
Main():void =
    var Tried:int = 0
    if (G := 1 | 5, set Tried += 1, G > 3):
        Print("{G}, Tried = {Tried}")
```

Run in the playground

```
5, Tried = 1
```

```
Gems():int = 1 | 5 | 10

Main():void =
    for (G := Gems(), G > 3):
        Print("a gem worth {G}")
```

Run in the playground

```
a gem worth 5
a gem worth 10
```

The Rule of For

A `for` visits each choice in order, skips the ones whose filters fail, and gives back the array of its body's values.

The Rule of Choices

A choice `a | b` has `a`'s values, then `b`'s, and so does a call of a recipe whose body is one. A `for` takes every one; an `if` takes the first that makes its condition succeed, and takes back the tries that failed.

```
64
```

```
Main():void =
    var N:int = 1
    loop:
        set N *= 2
        if (N > 50) then break
    Print("{N}")
```

Run in the playground

```
Main():void =
    for (X := 1..3) { break }
```

Run in the playground

Rejected: `break is only allowed inside loop`

```
14
0
```

```
FirstBig(A:[]int):int =
    for (X : A, X > 10) { return X }
    0

Main():void =
    Print("{FirstBig(array{3, 14, 25})}")
    Print("{FirstBig(array{1, 2})}")
```

Run in the playground

The Rule of Loop

A `loop` repeats until a `break` (or a `return`) leaves it. A `for` needs neither: it stops when it runs out of choices.

```
Countdown(N:int):void =
    if (N > 0):
        Print("{N}")
        Countdown(N - 1)
    else:
        Print("liftoff!")

Main():void = Countdown(3)
```

Run in the playground

```
3
2
1
liftoff!
```

```
Fact(N:int):int =
    if (N <= 1) then 1 else N * Fact(N - 1)

Main():void = Print("{Fact(4)}")
```

Run in the playground

```
24
```

```
Sum(A:[]int, I:int):int =
    if (X := A[I]):
        X + Sum(A, I + 1)
    else:
        0

Main():void =
    Print("{Sum(array{5, 10, 20}, 0)}")
```

Run in the playground

```
35
```

```
35
```

```
Sum(A:[]int):int =
    var Total:int = 0
    for (X : A) { set Total += X }
    Total

Main():void =
    Print("{Sum(array{5, 10, 20})}")
```

Run in the playground

```
8 squared is over 50
```

```
From(N:int):int = N | From(N + 1)

Main():void =
    if (X := From(1), X * X > 50):
        Print("{X} squared is over 50")
```

Run in the playground

The Rule of Recursion

A recipe may call itself, if each call is on a smaller problem and some question finally stops it. In Verse, a failure is often that question. A recipe whose choice calls itself may go on forever: whoever asks for its values stops it.

6

Maybe

```
Main():void =
    Box:?int = option{42}
    if (X := Box?):
        Print("the box holds {X}")
```

Run in the playground

```
the box holds 42
```

```
Main():void =
    Empty:?int = false
    if (Y := Empty?):
        Print("{Y}")
    else:
        Print("empty box")
```

Run in the playground

```
empty box
```

```
Main():void =
    Box:?int = option{42}
    X:int = Box
```

Run in the playground

Rejected: `expected int, found ?int`

```
Find(L:[]string, Want:string)<decides>:int =
    var Where:int = -1
    for (I -> G : L, G = Want, Where < 0):
        set Where = I
    Where >= 0
    Where

Main():void =
    Gems := array{"ruby", "opal"}
    Spot:?int = option{Find[Gems, "opal"]}
    None:?int = option{Find[Gems, "jade"]}
    if (S := Spot?) { Print("opal at {S}") }
    if (not None?) { Print("no jade") }
```

Run in the playground

```
opal at 1
no jade
```

```
Main():void =
    var Pocket:?string = false
    set Pocket = option{"key"}
    if (K := Pocket?) { Print("a {K}") }
```

Run in the playground

```
a key
```

```
Main():void =
    var Pocket:?string = false
    Print(Pocket? or "nothing")
    set Pocket = option{"key"}
    Print(Pocket? or "nothing")
```

Run in the playground

```
nothing
key
```

The Rule of the Box

An option `?t` is a box: `option{X}` holds `X`, `false` is empty. `Box?` opens it, and fails when it is empty. `option{…}` turns success into a full box and failure into an empty one.

```
Main():void =
    Prices:[string]int =
        map{"map" => 30, "key" => 5}
    if (P := Prices["key"]):
        Print("a key costs {P}")
    if (P := Prices["boat"]):
        Print("{P}")
    else:
        Print("no boats here")
```

Run in the playground

```
a key costs 5
no boats here
```

```
    set Bag["gem"] = 1
    set Bag["coin"] = 4
```

Run in the playground

```
    Print("{Thing}: {Count}")
```

```
gem: 1
```

```
Main():void =
    var Bag:[string]int = map{"coin" => 3}
    if (set Bag["coin"] += 1,
        C := Bag["coin"]):
        Print("now {C}")
    if (set Bag["gem"] += 1):
        Print("?")
    else:
        Print("no gems to add to")
```

Run in the playground

```
now 4
no gems to add to
```

```
Main():void =
    Prices:[string]int =
        map{"map" => 30, "key" => 5}
    Print("{Prices.Length} things for sale")
```

Run in the playground

```
2 things for sale
```

The Rule of the Map

A map `[k]v` finds a value by its key, and fails when the key is missing. `set M[K] = V` adds or replaces; `set M[K] += V` needs the key to be there.

7

Shapes of Things

```
coin := struct:
    Metal:string = "gold"
    Worth:int

Main():void = {}
```

Run in the playground

```
coin := struct:
    Metal:string = "gold"
    Worth:int

Main():void =
    A := coin{Worth := 10}
    B := coin{Metal := "silver", Worth := 2}
    Print("{A.Metal}, worth {A.Worth}")
    Print("{B.Metal}, worth {B.Worth}")
```

Run in the playground

```
gold, worth 10
silver, worth 2
```

```
coin := struct:
    Metal:string = "gold"
    Worth:int

Main():void =
    A := coin{Worth := 10}
    if (A = coin{Worth := 10}):
        Print("same coin")
```

Run in the playground

```
same coin
```

```
coin := struct:
    Worth:int

Main():void =
    var C:coin = coin{Worth := 1}
    D := C
    set C.Worth = 9
    Print("{C.Worth} {D.Worth}")
```

Run in the playground

```
9 1
```

```
door := enum:
    Open
    Locked
    Stuck

Main():void = {}
```

Run in the playground

```
find the key
push harder
```

```
door := enum:
    Open
    Locked
    Stuck

Say(D:door):string =
    case (D):
        door.Open => "come in"
        door.Locked => "find the key"
        door.Stuck => "push harder"

Main():void =
    Print(Say(door.Locked))
    Print(Say(door.Stuck))
```

Run in the playground

```
door := enum:
    Open
    Locked

Say(D:door):string =
    case (D):
        door.Open => "come in"

Main():void = Print(Say(door.Open))
```

Run in the playground

Rejected: `a case without _`

The Rule of Shapes

A `struct` is a value made of named fields; two structs with equal fields are equal. An `enum` is a type with a fixed list of values, and a `case` must say what to do with each of them.

```
player := class:
    Name:string
    var Score:int = 0

Main():void = {}
```

Run in the playground

```
player := class:
    Name:string
    var Score:int = 0
    Win(Points:int)<transacts>:void =
        set Score += Points
    Show()<reads>:string =
        "{Name} has {Score}"

Main():void =
    P := player{Name := "Ada"}
    Same := P
    P.Win(5)
    Same.Win(3)
    Print(P.Show())
```

Run in the playground

```
Ada has 8
```

```
chest := class:
    var Gold:int = 0

Main():void =
    C := chest{}
    if (set C.Gold = 100, C.Gold > 500):
        Print("rich")
    Print("{C.Gold}")
```

Run in the playground

```
0
```

```
hero := class<abstract>:
    Name:string
    Cheer():string

knight := class(hero):
    Cheer<override>():string = "For honor!"

wizard := class(hero):
    Cheer<override>():string = "Ta-da!"

Main():void =
    Party:[]hero = array{
        knight{Name := "Ada"},
        wizard{Name := "Lin"}}
    for (H : Party):
        Print("{H.Name}: {H.Cheer()}")
```

Run in the playground

```
Ada: For honor!
Lin: Ta-da!
```

```
Ada is a knight
```

```
hero := class<abstract>:
    Name:string
knight := class(hero):
    Sword:int = 1
wizard := class(hero):
    Spells:int = 3

Main():void =
    Party:[]hero = array{
        knight{Name := "Ada"},
        wizard{Name := "Lin"}}
    for (H : Party, K := knight[H]):
        Print("{K.Name} is a knight")
```

Run in the playground

The Rule of Objects

A `class` makes objects. Every name for an object sees the same object, and its `var` fields may change. An object always knows its own class, and `T[X]` asks whether `X` is a `T`.

8

Promises Kept

```
Main()<suspends>:void =
    Print("dig")
    Sleep(1.0)
    Print("found it")
```

Run in the playground

```
dig
found it
```

```
Rest():void =
    Sleep(1.0)

Main():void = Rest()
```

Run in the playground

Rejected: `it can only be called in a <suspends> function`

```
Dig(Name:string, Holes:int)<suspends>:void =
    for (I := 1..Holes):
        Print("{Name} digs hole {I}")
        Sleep(1.0)

Main()<suspends>:void =
    sync:
        Dig("Ada", 2)
        Dig("Lin", 3)
    Print("all dug")
```

Run in the playground

```
Ada digs hole 1
Lin digs hole 1
Ada digs hole 2
Lin digs hole 2
Lin digs hole 3
all dug
```

```
first came the storm
```

```
Search()<suspends>:string =
    Sleep(3.0)
    "treasure"

Storm()<suspends>:string =
    Sleep(2.0)
    "storm"

Main()<suspends>:void =
    First := race:
        Search()
        Storm()
    Print("first came the {First}")
```

Run in the playground

```
pack up the shovel
first came the storm
```

```
Search()<suspends>:string =
    defer:
        Print("pack up the shovel")
    Sleep(3.0)
    "treasure"

Storm()<suspends>:string =
    Sleep(2.0)
    "storm"

Main()<suspends>:void =
    First := race:
        Search()
        Storm()
    Print("first came the {First}")
```

Run in the playground

```
Lin's map
Ada's map
```

```
Fetch(Who:string, T:float)<suspends>:string =
    Sleep(T)
    "{Who}'s map"

Main()<suspends>:void =
    A := spawn{Fetch("Ada", 2.0)}
    L := spawn{Fetch("Lin", 1.0)}
    Print(L.Await())
    Print(A.Await())
```

Run in the playground

```
wait for a ship
```

```
Main()<suspends>:void =
    Print("wait for a ship")
    Sleep(Inf)
    Print("never")
```

Run in the playground

The Rule of Turns

A `<suspends>` recipe may wait. A `sync` waits for all its arms, a `race` for the first, and `spawn` starts a task to `Await` later. Tasks take turns on a clock of ticks, always in the same order, so a program with tasks still has one answer.

```
40
```

```
Worth(Gems:int)<computes>:int =
    Gems * 10

Main():void = Print("{Worth(4)}")
```

Run in the playground

```
chest := class:
    var Gold:int = 0

Rob(C:chest)<reads>:void =
    set C.Gold = 0

Main():void = Rob(chest{})
```

Run in the playground

Rejected: `Rob sets a var field, which its effects (<reads>) do not allow`

The Rule of Promises

A recipe's effects are promises: `<decides>` may fail, `<suspends>` may wait, `<computes>` touches no changing thing, `<reads>` only looks, `<allocates>` only makes new things, `<transacts>` may do it all. Verse refuses a recipe that breaks its promise.

```
Main():void =
    var Gold:int = 9223372036854775807
    Print("so much gold")
    set Gold += 1
    Print("even more")
```

Run in the playground

```
so much gold
```

```
Main():void =
    Print("the map is a fake")
    Err()
    Print("unreachable")
```

Run in the playground

```
the map is a fake
```

```
Unlock(Code:int)<decides>:void =
    Code = 1234

Main()<decides>:void =
    Print("trying the code")
    Unlock[1111]
    Print("unlocked!")
```

Run in the playground

```
trying the code
```

The Rule of Errors

Failure is a polite "no" that a question can catch.
An error, such as an overflow or `Err()`, is a real
mistake: nothing catches it, and the program stops.

The Rule of Trust

If Verse accepts your program, it has one answer, its failed guesses leave no trace, its recipes without `<decides>` never fail, its types are kept, and its promises about objects hold. Only overflow, `Err()` and endless loops are left to watch for.

A

The Rules

The Rule of the Braces

To show a value, put it in braces
inside a string, and print the string.

The Rule of Types

Every value has a type: `int`, `float`, `logic`, `char`, `string`, an array type such as `[]int`, or a tuple type such as `tuple(int, string)`. An array's elements all share one type; a tuple's parts each have their own.

The Rule of Names

A name stands for one value for as long as it lives. Only a `var` may change, and only by `set`.

The Rule of Recipes

A function names its parameters, says what type it gives back, and gives back the value of its last line. A `void` function is used for what it does.

The Rule of Failure

A question in Verse succeeds or fails. Anything that might fail must sit in a failure context, such as the condition of an `if`.

The Rule of the Question Mark

`false` is a value, not a failure. To turn a `logic` into a question, add `?`: `X?` succeeds when `X` is `true` and fails when it is `false`.

The Rule of Brackets

A function that may fail is marked `<decides>` and is called with square brackets, `F[X]`. A function without `<decides>` is called with parentheses, `F(X)`, and never fails.

The Rule of the Safe Guess

When a question fails, everything it did is taken back: every `set` and every `Print`. When it succeeds, everything it did is kept.

The Rule of Not, Or and And

`not` succeeds when its inside fails, and never keeps its inside's work. `or` tries the left side, and if it fails, takes it back and tries the right. `and` needs both sides to succeed.

The Rule of For

A `for` visits each choice in order, skips the ones whose filters fail, and gives back the array of its body's values.

The Rule of Choices

A choice `a | b` has `a`'s values, then `b`'s, and so does a call of a recipe whose body is one. A `for` takes every one; an `if` takes the first that makes its condition succeed, and takes back the tries that failed.

The Rule of Loop

A `loop` repeats until a `break` (or a `return`) leaves it. A `for` needs neither: it stops when it runs out of choices.

The Rule of Recursion

A recipe may call itself, if each call is on a smaller problem and some question finally stops it. In Verse, a failure is often that question. A recipe whose choice calls itself may go on forever: whoever asks for its values stops it.

The Rule of the Box

An option `?t` is a box: `option{X}` holds `X`, `false` is empty. `Box?` opens it, and fails when it is empty. `option{…}` turns success into a full box and failure into an empty one.

The Rule of the Map

A map `[k]v` finds a value by its key, and fails when the key is missing. `set M[K] = V` adds or replaces; `set M[K] += V` needs the key to be there.

The Rule of Shapes

A `struct` is a value made of named fields; two structs with equal fields are equal. An `enum` is a type with a fixed list of values, and a `case` must say what to do with each of them.

The Rule of Objects

A `class` makes objects. Every name for an object sees the same object, and its `var` fields may change. An object always knows its own class, and `T[X]` asks whether `X` is a `T`.

The Rule of Turns

A `<suspends>` recipe may wait. A `sync` waits for all its arms, a `race` for the first, and `spawn` starts a task to `Await` later. Tasks take turns on a clock of ticks, always in the same order, so a program with tasks still has one answer.

The Rule of Promises

A recipe's effects are promises: `<decides>` may fail, `<suspends>` may wait, `<computes>` touches no changing thing, `<reads>` only looks, `<allocates>` only makes new things, `<transacts>` may do it all. Verse refuses a recipe that breaks its promise.

The Rule of Errors

Failure is a polite "no" that a question can catch. An error, such as an overflow or `Err()`, is a real mistake: nothing catches it, and the program stops.

The Rule of Trust

If Verse accepts your program, it has one answer, its failed guesses leave no trace, its recipes without `<decides>` never fail, its types are kept, and its promises about objects hold. Only overflow, `Err()` and endless loops are left to watch for.